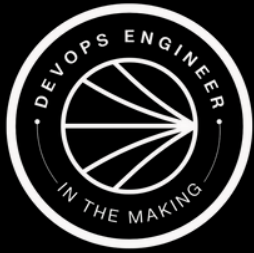
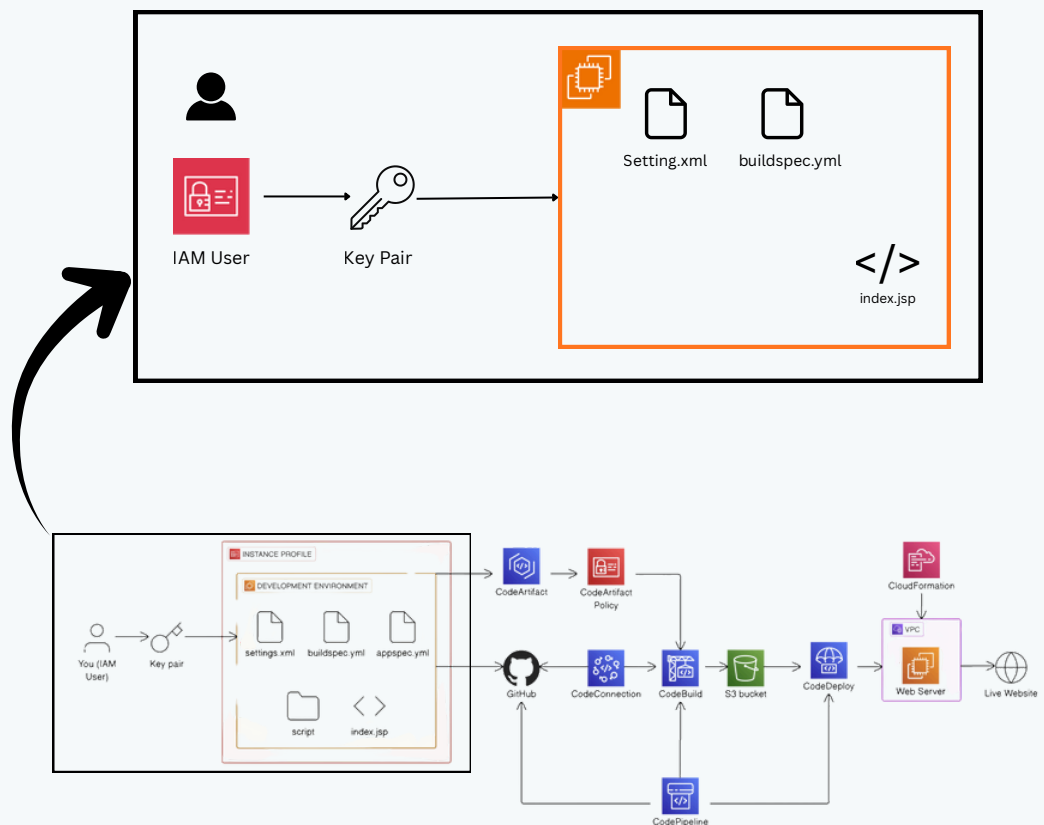


2025



DevOps Project

Setting Up a Web App in the Cloud



Nikko Cajayon
Community Lead



TABLE OF CONTENTS

1

INTRODUCTION

Introduction
System Architecture
Tools and services

2

SET UP AN IAM ACCOUNT

Understanding IAM policy
Importance of priviledge permisison

3

LAUNCH AN EC2 INSTANCE

Provisioning EC2 instance server
Network Configuration

4

SETTING UP VS CODE

Connecting sucurely through .pem file
accessing server through terminal

5

CREATING APPLICATION

Installing Apache Maven
Installing Java Correto 8

6

VSCODE AND EC2 INSTANCE CONFIGURATION

secure connection using SSH Secure Shell
IDE on the cloud Set-up



1 INTRODUCTION

In this first part of our 7-Day DevOps Project, we will establish the essential Cloud Foundation and the Compute Layer for our application. The primary objective is to provision a reliable and secure deployment target by setting up a basic web application on an **AWS EC2 instance**.

This process involves crucial foundational steps: configuring the networking (**Security Groups**) to allow public HTTP access, and using a User Data Script to ensure the server infrastructure is correctly initialized with a basic web server. This validates that our future deployment target is fully operational from Day 1.

Crucially, to enable automation and security, we will implement the necessary IAM roles (following the principle of least privilege) and associate them with our EC2 instance. This configuration allows the instance to securely communicate with the AWS CodeDeploy service, which is a non-negotiable prerequisite for building the robust CI/CD pipeline we will develop over the coming days.

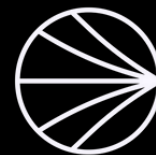


OBJECTIVE

- 🖥️ Launch an EC2 instance.
- 🔌 Use VS Code to set up a remote SSH connection to your EC2 instance.
- 🍷 Install Maven and Java and generate a basic web app.
- 💎 Edit code without VS Code - you'll see why IDEs are so well loved after

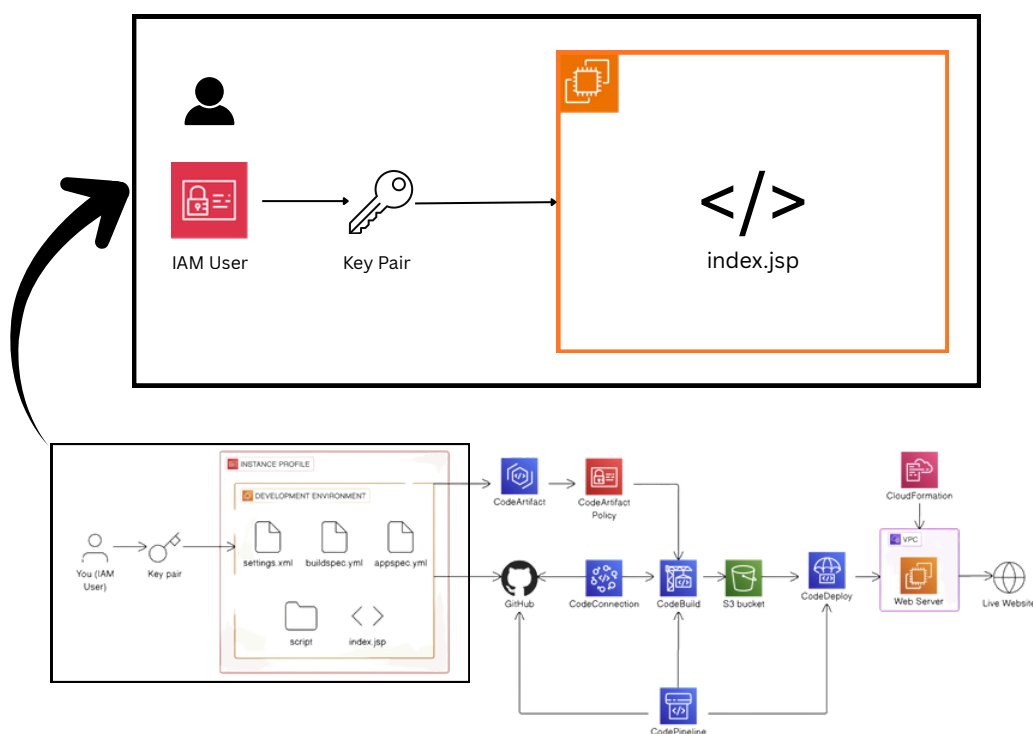
PREREQUISITE

Concept	Description
Amazon EC2 (Elastic Compute Cloud)	AWS service providing scalable compute capacity (virtual servers) in the cloud.
Key Pair	A set of security credentials, consisting of a public key (stored by AWS) and a private key (stored by you).
IAM (Identity and Access Management)	AWS service that manages access to AWS resources securely, defining who can access what.
VS Code (Visual Studio Code)	A free, open-source code editor.



ARCHITECTURE DIAGRAM

This project utilizes a hybrid architecture, initially focusing on secure manual access (Day 1) before transitioning into a fully automated, end-to-end Continuous Integration and Continuous Delivery (CI/CD) pipeline (Day 2 - Day 7).



Component	Description
IAM User	The developer/admin identity. Access is managed by IAM policies and enforces the principle of least privilege. Code is managed locally using VS Code.
Key Pair	Used to establish a secure, encrypted SSH connection to the EC2 instance for initial setup and troubleshooting.
EC2 Instance / Web Server	The target compute resource hosting the web app. It is provisioned with an IAM Role allowing it to communicate with services like CodeDeploy.



2

SET UP AN IAM ACCOUNT

The initial setup requires an AWS Identity and Access Management (IAM) user with sufficient permissions to create, configure, and manage all the AWS services used throughout this 7-day project (EC2, CodeBuild, CodePipeline, etc.).

Since this project requires setup across several services (EC2, CodePipeline, CodeBuild, S3, IAM, CloudFormation), the user must have broad administrative capabilities to build the pipeline.

The screenshot displays the AWS IAM console for a user named 'DevOps-Engineer'. The 'Summary' section shows the ARN as 'arn:aws:iam::210806348419:user/DevOps-Engineer', created on 'October 15, 2025, 20:30 (UTC+08:00)', with console access 'Enabled without MFA' and a last sign-in 'Today'. The 'Permissions' tab is active, showing 9 policies. A table lists these policies, all of which are 'AWS managed'.

Policy name	Type
AmazonEC2FullAccess	AWS managed
AmazonQDeveloperAccess	AWS managed
AmazonS3FullAccess	AWS managed
AWSBillingReadOnlyAccess	AWS managed
AWSCloudFormationFullAccess	AWS managed
AWSCodeBuildAdminAccess	AWS managed
AWSCodeDeployFullAccess	AWS managed
AWSCodePipeline_FullAccess	AWS managed
IAMFullAccess	AWS managed



IAM ACCOUNT PERMISSION

IAMFullAccess	Required to create the necessary IAM roles for the EC2 instance, CodeBuild, CodeDeploy, and CodePipeline.
Amazon EC2FullAccess	Required to launch, configure, and manage the target EC2 instance.
AWS CodeDeployFullAccess	Required to create the CodeDeploy Application and Deployment Group (Day 5).
AWS CodeBuildAdminAccess	Required to create and configure the CodeBuild project (Day 4).
AWS CodePipeline_FullAccess	Required to orchestrate and manage the full CI/CD pipeline (Day 7).
AWS CodePipelineFullAccess	Required to manage S3 Buckets used for artifacts (CodeBuild output) and CodeDeploy revisions.
AWS CloudFormationFullAccess	Required to create and manage the stack in Part 6 (IaC). Export to Sheets



3

LAUNCH AN EC2 INSTANCE

The initial setup requires an AWS Identity and Access Management (IAM) user with sufficient permissions to create, configure, and manage all the AWS services used throughout this 7-day project (EC2, CodeBuild, CodePipeline, etc.).

The screenshot shows the 'Launch an instance' page in the AWS Management Console. The page is divided into several sections:

- Name and tags:** A text input field contains 'DevOps-nyx-server'. There is a link to 'Add additional tags'.
- Application and OS Images (Amazon Machine Image):** A search bar with the text 'Search our full catalog including 1000s of application and OS images'. Below it is a 'Quick Start' section with icons for various operating systems: Amazon Linux, macOS, Ubuntu, Windows, Red Hat, SUSE Linux, and Debian. To the right is a 'Browse more AMIs' link.
- Amazon Machine Image (AMI):** A dropdown menu shows 'Amazon Linux 2023 kernel-6.1 AMI' with its ID 'ami-088d74dfe9802f14'.
- Summary:** A sidebar on the right contains configuration details:
 - Number of instances:** 1
 - Software Image (AMI):** Amazon Linux 2023 AMI 2023.9.2...read more
 - Virtual server type (instance type):** t2.micro
 - Firewall (security group):** New security group
 - Storage (volumes):** 1 volume(s) - 8 GiB
- Buttons:** At the bottom right of the summary section are 'Cancel' and 'Launch instance' buttons, along with a 'Preview code' link.

WHAT IS EC2

is a service that lets you rent and use virtual computers in the cloud. They're like your personal computers, but they exist on the internet instead of being physically in front of you. You can create, customize, and use these computers for all different reasons, from running applications to hosting websites.



KEY PAIR

A key pair is a secure way to access your virtual computer (EC2 instance) on AWS. It's like having a special key that only you can use to unlock your server. This ensures that no one else can access your virtual machine without your permission.

▼ **Key pair (login)** [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair.

Key pair name - required

DevOps-keypair

Q

Proceed without a key pair (Not recommended) Default value

NyxAuroraServer
Type: rsa

NW-Aurora-App
Type: rsa

NyxAuroraApp
Type: rsa

Training
Type: rsa

DevOps-keypair
Type: rsa ✓

Auto-assign public IP [Info](#)

NETWORK SETUP - SSH

SSH is also a type of network traffic. Once SSH has authorized you, it'll set up a secure connection between you and the EC2 instance. All data transferred (including your commands and the responses from the instance) gets encrypted. This encryption makes SSH an ideal method for working with virtual servers!



4 SETTING UP VS CODE

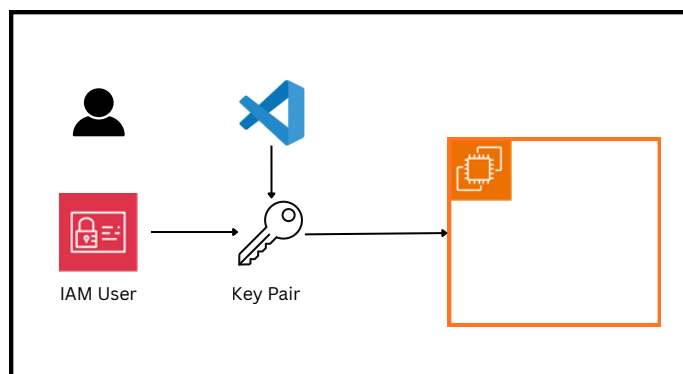
Now that your EC2 instance is up and running, how are we going to get inside your instance and set up a web app?

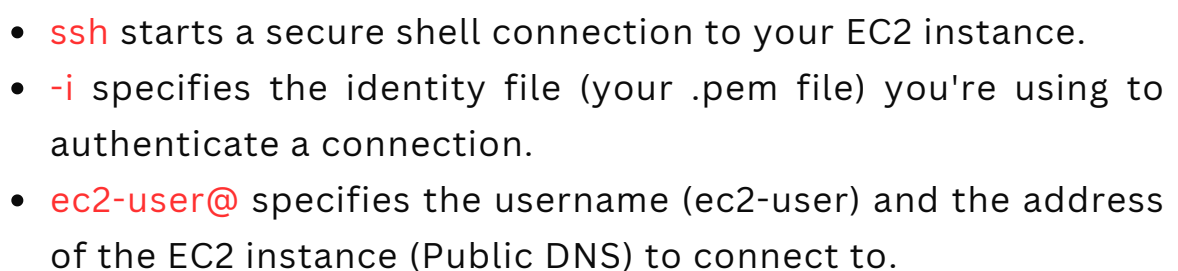
We'll use Visual Studio Code (VSCode) to connect with your instance, so you can create and edit your web app's code.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE  CODE REFERENCE LOG
processed file: keypair-devops.pem
Successfully processed 1 files; Failed processing 0 files
(base) PS C:\Users\USER\Documents\DevOps> cmd /c 'icacls "keypair-devops.pem" /grant:r "niks\user":(F)
processed file: keypair-devops.pem
Successfully processed 1 files; Failed processing 0 files
(base) PS C:\Users\USER\Documents\DevOps>
```

We're changing the permissions of your .pem file so that you have access to it when you connect to your EC2 instance later. Blocking out everyone else keeps your .pem file i.e. your secret key secure.

- /reset to remove default permission settings on the file
- /grant:r "%USERNAME%:R" to give the current user (that's you!) read access to your secret key
- /inheritance:r to make sure changes in the permissions of other files and the DevOps folder won't change the permission settings for this file.







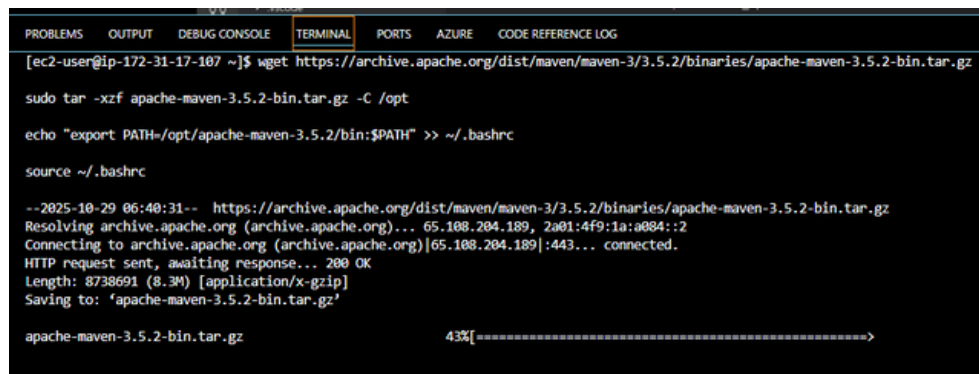
INSTALL APACHE MAVEN AND AMAZON CORRETTO 8

In this step, you're going to:

1. Install Apache Maven on your EC2 instance.
2. Install Amazon Corretto 8, a version of Java.
3. Verify the installations.

What is Apache Maven?

Apache Maven is a tool that helps developers build and organize Java software projects. It's also a package manager, which means it automatically download any external pieces of code your project depends on to work.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE CODE REFERENCE LOG
[ec2-user@ip-172-31-17-107 ~]$ wget https://archive.apache.org/dist/maven/maven-3/3.5.2/binaries/apache-maven-3.5.2-bin.tar.gz
sudo tar -xzf apache-maven-3.5.2-bin.tar.gz -C /opt
echo "export PATH=/opt/apache-maven-3.5.2/bin:$PATH" >> ~/.bashrc
source ~/.bashrc
--2025-10-29 06:40:31-- https://archive.apache.org/dist/maven/maven-3/3.5.2/binaries/apache-maven-3.5.2-bin.tar.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:1a:a084::2
Connecting to archive.apache.org (archive.apache.org)[65.108.204.189]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 8738691 (8.3M) [application/x-gzip]
Saving to: 'apache-maven-3.5.2-bin.tar.gz'
apache-maven-3.5.2-bin.tar.gz 43%[=====
```

wget https://archive.apache.org/dist/maven/maven-3/3.5.2/binaries/apache-maven-3.5.2-bin.tar.gz

sudo tar -xzf apache-maven-3.5.2-bin.tar.gz -C /opt

echo "export PATH=/opt/apache-maven-3.5.2/bin:\$PATH" >> ~/.bashrc

source ~/.bashrc



5

CREATING APPLICATION

We've assembled both Maven and Java into our EC2 instance. Now let's cut straight to generating the web app!

In this step, you're going to:

1. Run Maven commands in your terminal to generate a Java web app.

```
[ec2-user@ip-172-31-17-107 ~]$ mvn archetype:generate \
-DgroupId=com.nextwork.app \
-DartifactId=nextwork-web-project \
-DarchetypeArtifactId=maven-archetype-webapp \
-DinteractiveMode=false
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.pom (3.9 kB at 5.1 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/22/maven-plugins-22.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/22/maven-plugins-22.pom (13 kB at 114 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/21/maven-parent-21.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/21/maven-parent-21.pom (26 kB at 258 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/10/apache-10.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/10/apache-10.pom (15 kB at 197 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.jar (25 kB at 181 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/2.4/maven-install-plugin-2.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/2.4/maven-install-plugin-2.4.pom (6.4 kB at 51 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/23/maven-plugins-23.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/23/maven-plugins-23.pom (9.2 kB at 87 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/22/maven-parent-22.pom
```

```
mvn archetype:generate \
-DgroupId=com.nextwork.app \
-DartifactId=nextwork-web-project \
-DarchetypeArtifactId=maven-archetype-webapp \
-DinteractiveMode=false
```



```
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-webapp:1.0
[INFO] -----
[INFO] Parameter: basedir, Value: /home/ec2-user
[INFO] Parameter: package, Value: com.nextwork.app
[INFO] Parameter: groupId, Value: com.nextwork.app
[INFO] Parameter: artifactId, Value: nextwork-web-project
[INFO] Parameter: packageName, Value: com.nextwork.app
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/ec2-user/nextwork-web-project
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 24.743 s
[INFO] Finished at: 2025-10-29T07:03:13Z
[INFO] Final Memory: 20M/92M
[INFO] -----
[ec2-user@ip-172-31-17-107 ~]$
```

- DartifactId=nextwork-web-project** names your project
- DarchetypeArtifactId=maven-archetype-webapp** specifies that you're creating a web application.
- DinteractiveMode=false** runs the command without pausing for user input, so Maven will go ahead and install everything without waiting for your confirmation.



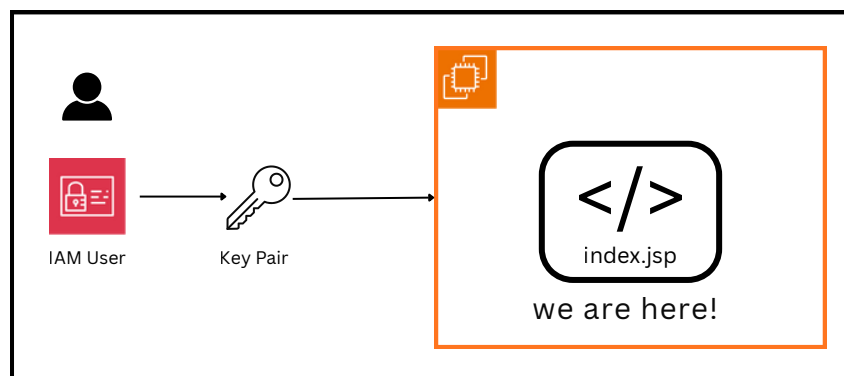
6

VSCODE AND EC2 INSTANCE CONFIGURATION

You might be thinking... have I really just set up a web app?
Where is it, how can I actually see it?

In this step, we'll going to:

1. Install an extension in VSCode.
2. Use the extension to set up a connection between VSCode and your EC2 instance.
3. Explore and edit your Java web app's files using VSCode.



Aren't I already connected to my EC2 instance with SSH?

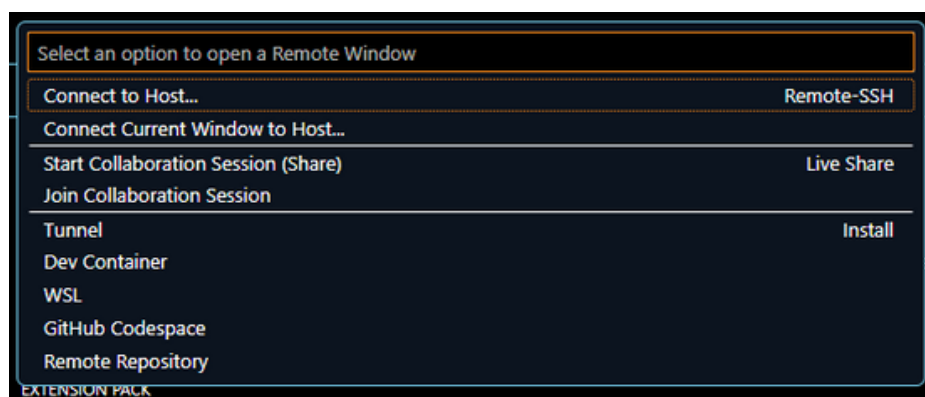
Yes, connecting with SSH in VSCode's terminal lets you send text commands to your EC2 instance, but you don't get all the benefits of having an IDE like VSCode.

When you connect VSCode itself to your EC2 instance (not just your terminal), you unlock VSCode's IDE features (like file navigation and code editing) directly on your EC2 instance. This will make it so much easier for us to edit and manage our web app for the rest of this Devops project.



SETTING UP REMOTE-SSH

The Remote - SSH extension in VSCode lets you connect directly via SSH to another computer securely over the internet. This lets you use VSCode to work on files or run programs on that server as if you were doing it on your own computer, which will come in handy when we edit the web app in your EC2 instance!



What is SSH Host?

An SSH Host is the computer or server you're connecting to using SSH. It's the target location where you want to run commands or manage files; in our case, the SSH Host is the EC2 instance we created.



7

JAVA WEB APP

After connecting to our server using SSH, we can now access and edit our web-app source code. this is basically building straight from cloud.

The screenshot shows an IDE interface with a dark theme. On the left, the 'EXPLORER' panel displays a project structure for 'NEXTWORK-WEB-PROJECT [SSH: EC2-13-250-113-...]'. The structure includes a 'src/main' directory with subdirectories 'resources', 'webapp', and 'WEB-INF'. The 'web.xml' file is selected under 'WEB-INF'. The main editor area shows the content of 'web.xml' with the following XML code:

```
1 <!DOCTYPE web-app PUBLIC
2 "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3 "http://java.sun.com/dtd/web-app_2_3.dtd" >
4
5 <web-app>
6   <display-name>Archetype Created Web Application</display-name>
7 </web-app>
8
```



PROJECT SUMMARY

Conclusion: Establishing the Target Compute Base

Day 1 successfully established the target compute environment and the necessary security and access foundation for the entire 7-day project.

Key Achievements:

1. Secure Compute Layer: A publicly accessible EC2 instance was provisioned, secured by a dedicated Security Group and bootstrapped with Nginx via a User Data script (initial IaC practice).
2. Deployment Readiness (IAM): The critical EC2-CodeDeploy-Role was attached to the instance, ensuring the target server has the necessary Identity and Access Management (IAM) permissions to communicate with AWS CodeDeploy and receive automatic application revisions later in the week.
3. Developer Experience (DX): Secure connectivity was verified via SSH, allowing for remote development directly on the server's file system through VS Code. This simulates a modern, efficient cloud development workflow.



Nikko Cajayon

NextWork Community Lead
Director for SkillBuilder
CTO of AWS Cloud Club

Here at NextWork We are
Transforming the Way World Learns